

Day 2 Evidence Workbooklet — Instructor Key

Variables, Expressions, and State

Engineering practice → current state → expression value → trace history → support next step

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/50		Mastery check		secure / developing / needs support

Your team is checking a small cooling-pump calibration record before the result is copied into a lab log. The pump has a target flow rate. A notebook is used to track the current measured flow, calculate planned corrections, and record when a planned value becomes the current value.

The problem today is state: what value is stored right now, what value was only planned or calculated, and which checkpoint should a lab note cite?

Engineering task	Keep a reliable calibration record while a pump-flow setting is checked, planned, approved, and then updated.
Computational move	Track current state after each assignment and separate state history from the value of one expression.
Python focus	variable assignment, arithmetic expression, overwrite, calculated variable, current value after each cell.
Evidence to keep	variable role, value after each checkpoint, expression meaning, overwritten value, and one corrected lab-log sentence.

Day 2 state rules

1. A variable's current value is the most recent value assigned to that name.
2. A later assignment can overwrite an earlier value, but the old value may still matter in the history.
3. A calculated variable does not update automatically when another variable changes later; it updates only when a new assignment runs.
4. A planned value is not the same as the recorded current value until the code assigns it to the current-value variable.
5. A reliable lab note names the checkpoint, not only the number.

1 Read the notebook as a state-history record

[recognize](#)[predict](#)

Use the scenario above. First identify what each variable is supposed to represent. Do not branch, loop, or write a full program.

```
1 # Cell 1: baseline pump record
2 pump_id = "P-12"
3 target_flow_lpm = 18.0
4 flow_lpm = 16.8
5 gap_lpm = target_flow_lpm - flow_lpm
6
7 # Cell 2: calculate a planned correction
8 adjust_lpm = 0.9
9 planned_flow_lpm = flow_lpm + adjust_lpm
10 planned_gap_lpm = target_flow_lpm - planned_flow_lpm
11
12 # Cell 3: approve the planned correction
13 flow_lpm = planned_flow_lpm
14 gap_lpm = planned_gap_lpm
15
16 # Cell 4: test one more small correction
17 adjust_lpm = 0.2
18 planned_flow_lpm = flow_lpm + adjust_lpm
19 gap_lpm = target_flow_lpm - planned_flow_lpm
20
21 # Cell 5: record only after supervisor approval
22 flow_lpm = planned_flow_lpm
23 gap_lpm = target_flow_lpm - flow_lpm
```

Pts.	Prompt	My evidence
1	Which variable stores the pump label rather than a number?	Key: pump_id
1	Which variable stores the target flow rate that the pump should approach?	Key: target_flow_lpm
1	Which variable stores the current recorded pump flow?	Key: flow_lpm
1	Which variable stores a planned flow value before it becomes the current recorded value?	Key: planned_flow_lpm

2 Trace the first planned correction

trace predict

Complete the state table for the first two cells. If a variable does not exist yet, write **not defined**.

Pts.	Checkpoint	flow_lpm	gap_lpm	planned_flow_lpm	planned_gap_lpm	Reason for change
4	After Cell 1	Key: 16.8	Key: 1.2	Key: not defined	Key: not defined	Key: Base-line gap: 18.0 - 16.8.
4	After Cell 2	Key: 16.8	Key: 1.2	Key: 17.7	Key: 0.3	Key: Planned correction calculated; current flow not reassigned.

Pts.	Prompt	My response
2	After Cell 2, why has planned_flow_lpm changed while flow_lpm has not changed?	Key: Cell 2 assigns to planned_flow_lpm, not flow_lpm.
2	After Cell 2, which value is the current recorded flow and which value is only the planned flow?	Key: Current: 16.8; planned: 17.7.

3 Trace approval and a second planned correction

[trace](#) [explain](#)

Now trace Cells 3–5. Watch for the difference between a planned value and a current recorded value.

Pts.	Checkpoint	flow_lpm	gap_lpm	planned_flow_lpm	planned_gap_lpm	Reason for change
3	After Cell 3	Key: 17.7	Key: 0.3	Key: 17.7	Key: 0.3	Key: Approval copies planned flow/gap into current variables.
4	After Cell 4	Key: 17.7	Key: 0.1	Key: 17.9	Key: 0.3	Key: New plan made. gap_lpm is based on planned flow; planned_gap_lpm remains stale.
3	After Cell 5	Key: 17.9	Key: 0.1	Key: 17.9	Key: 0.3	Key: Approved planned flow becomes current; gap recomputed from current flow.

Common trap to check

After Cell 4, `planned_flow_lpm` has a new planned value. The current recorded flow is still stored in `flow_lpm` until Cell 5 runs.

Pts.	Prompt	My response
2	After Cell 4, what makes the state record easy to misread?	Key: The notebook has a planned 17.9 and a 0.1 gap, but current <code>flow_lpm</code> is still 17.7.
2	After Cell 5, what changed that makes 17.9 the current recorded flow?	Key: <code>flow_lpm = planned_flow_lpm</code> ran at approval.

4 Evaluate expressions without losing the checkpoint

[predict](#) [explain](#)

For each expression or assignment, state the value it produces and whether it changes the current recorded flow.

Pts.	Line or expression	Value produced	Does it change flow_lpm? Explain.
1	target_flow_lpm - flow_lpm after Cell 1	Key: 1.2	Key: Expression/check value; it does not change flow_lpm.
1	flow_lpm + adjust_lpm during Cell 2	Key: 17.7	Key: Stored as planned flow; current flow stays 16.8.
1	flow_lpm = planned_flow_lpm in Cell 3	Key: copies 17.7	Key: Yes. This changes current flow_lpm.
1	target_flow_lpm - planned_flow_lpm in Cell 4	Key: 0.1	Key: No direct flow change; it is assigned to gap_lpm.

Pts.	Prompt	My response
2	Why is an expression value not enough by itself to prove what the current recorded flow is?	Key: A calculated expression can be planned/candidate only. Current state depends on which variable was assigned at that checkpoint.
2	Which assignment line is the clearest evidence that a planned value became the current recorded value?	Key: flow_lpm = planned_flow_lpm.

5 Debug a lab-log sentence

[debug](#) [test](#) [explain](#)

A teammate writes this note after Cell 4:

Lab-log sentence to debug

The pump flow is 17.9 L/min because the notebook recalculated the gap as 0.1 L/min.

Pts.	Prompt	My response
2	What part of the teammate's sentence is based on a real calculation?	Key: After Cell 4, <code>planned_flow_lpm</code> is 17.9, so $18.0 - 17.9 = 0.1$.
2	What part of the sentence is unsafe or incomplete as a current-state claim?	Key: It calls 17.9 current before approval; after Cell 4, current <code>flow_lpm</code> is still 17.7.
2	Write one line you could run after Cell 4 to check the current recorded flow without changing it.	Key: <code>print(flow_lpm)</code> or bare <code>flow_lpm</code> .
2	Rewrite the lab-log sentence so it separates current recorded flow, planned flow, and approval checkpoint.	Key: After Cell 4, current <code>flow_lpm</code> is 17.7 L/min and planned flow is 17.9 L/min. After Cell 5 approval, <code>flow_lpm</code> stores 17.9 L/min.

State-history habit

When a notebook is used as an engineering record, write the checkpoint and variable name before copying the number. A good note sounds like: "After Cell ____, ____ stores ____."

6 Mastery check and support next step

[transfer](#)[explain](#)

Use your own trace evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example from the notebook where a variable was overwritten. Name the old value, the new value, and the checkpoint.	Key: flow_lpm: 16.8 to 17.7 in Cell 3, then 17.7 to 17.9 in Cell 5.
2	In one or two sentences, explain how state history helps prevent a bad engineering record.	Key: State history shows which value is current, planned, or stale at each checkpoint. That prevents copying a candidate calculation as approved/current.

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can trace the current value of a variable after several assignments and explain when a planned value becomes the recorded value.
☐	Developing: I can calculate some values, but I still need support separating planned values, current values, and overwritten values.
☐	Needs support: I need a smaller example before I can trace state history across several cells.

My next support move

My issue is mostly: setup / Python syntax / tracing / arithmetic / state history / confidence / time.

The first checkpoint where I got uncertain was: _____

My next small action is: ask a partner / ask instructor / rebuild the trace table / rerun one cell / try the recovery version.

Workbooklet target: I can track variable assignment, expression value, overwrite, and current state across a pump-calibration record before a result enters an engineering log.

Copyright © 2026 Lance L.A. White. All rights reserved.

Bridge Day 2 VIP Evidence Handoff

INSTRUCTOR KEY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Use current-state tracing to plan multi-step calculations, then finish the notebook's assigned-value and first input-to-output Studio programs.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: numeric literal, arithmetic expression, floor division, modulo, exponentiation, input call, int conversion, float conversion, f string, import statement, math library call

Carried authoring tools: string literal, assignment, print call, sequential execution, exact output

Supplied-code exposure only: none

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.3	Minutes Conversion	18-3-work / 18-3-transfer / 18-3-cold	arithmetic expression
18.4	Square Root Check	18-4-work / 18-4-transfer / 18-4-cold	f string
18.5	Kit Count Plan	18-5-work / 18-5-transfer / 18-5-cold	profile-level contract
18.6	Rectangle Diagonal	18-6-work / 18-6-transfer / 18-6-cold	f string
18.7	Grid Path Distance	18-7-work / 18-7-transfer / 18-7-cold	f string, input call, int conversion

Readiness evidence

1. Choose one activity and list its assigned or entered values, intermediate value, and final labeled output.

Model response: Credit an activity-specific chain whose intermediate value is genuinely used later, such as minutes remaining after full days or total capacity before unused items.

2. For Activity 18.7, write the read-convert-compute-format-print sequence and identify which three input results must become integers.

Model response: Read all three entries with input, convert each with int, compute total segments, feet, and miles, format the distance to two decimals with an f-string, then print.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.